Handout activities en fragement wisselen

Android studio

Deze handout bevat een kleine introductie met wat theorie en uitleg en het stappen plan voor het maken van de volgende dingen

- Activities
  - Hoe maak je een tweede
  - Intent – het wisselen tussen de twee
- Fragments
  - Containers voor elementen
  - Hoe maak je een fragment aan
  - Wisselen
  - Gevorderd – Layeren

Inhoud

1. Theorie uitleg 1

2. Activities 2

2.1. Hoe maak je een tweede activity 2

2.2. Intent – het wisselen tussen de twee 2

3. Fragments 4

3.1. Hoe maak je een fragment aan 4

3.2. Containers voor elementen 4

3.3. Wisselen tussen fragments 5

3.4. Gevorderd – Layeren 9

4. Hand-out afsluiting 9

1. Theorie uitleg

Activities zijn je schermen die je maakt als je een nieuw scherm aanmaakt wissel je tussen activities. Tijdens die wissel kan je informatie meegeven om zo een samenhangende app te maken. Deze wissel is een intent. Bij het wisselen van activity A naar activity B wordt A vernietigd.
Fragments zijn ook schermen maar kunnen als deelschermen functioneren. Ze worden vaak gebruikt voor gelaagde schermen waarbij elementen in lagen werken met elkaar. Denk aan Google maps waarbij de achtergrond de kaart is en de knoppen voorin zitten. Er zijn meerdere redenen om fragments te benutten maar de hoofd twee zijn het layeren van schermen en elementen, of het globaal houden van informatie tussen schermen.
In praktijk worden activities alleen nog gebruikt bij grote applicaties waarbij het nieuwe scherm een nieuwe inlaadt moment vereist van nieuwe data. Een kledingwinkel manager kan via zijn app de voorraad bekijken en het rooster van de medewerkers. Die twee schermen zouden activities zijn. Fragements worden dan gebruikt voor de detail schermen van de kledingstukken op de voorraad deel van de app aangezien de data van de voorraad hoogstwaarschijnlijk nog gebruikt gaat worden.

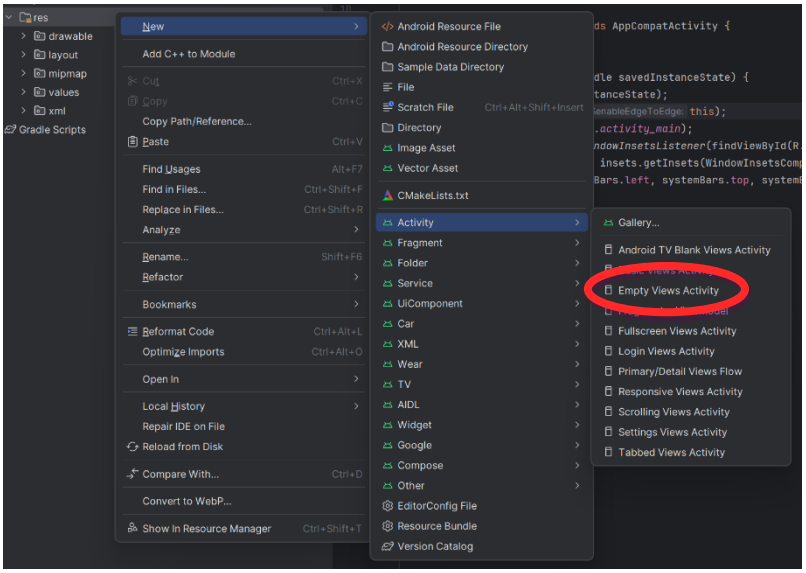
2. Activities

Activities kunnen binnen Android studio op twee plekken gevonden worden, de resources folder (res) binnen de map layouts en de code folder met daarin de Java bestanden.

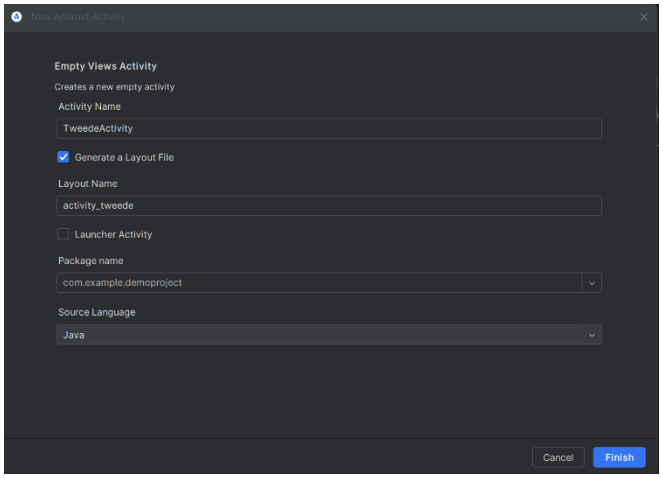
2.1. Hoe maak je een tweede activity

Om een extra activity te maken moet je de volgende stappen doorlopen.

- Klik met je rechtermuisknop op de res (resources) folder
- In dit menu, hoever over “New”
- In dit submenu, hoever over “Activity”
- In dit submenu bevinden templates voor verschillende activities, kies voor nu “Empty Views Activity”



Dit opent een menu, noem de nieuwe activity naar het onderwerp van de pagina en zorg ervoor een herkenbare unieke naam.



Zorg er ook voor dat de taal in Java blijft. Halverwege een switch maken naar Kotlin heeft soms het effect van je app corrupten.

Gefeliciteerd, je hebt nu een tweede activity gemaakt, deze kan je benutten net als iedere andere activity.

Om er naartoe te wisselen gaan we in het volgende stukje kijken naar intent.

2.2. Intent – het wisselen tussen de twee

Intent is een van twee manieren om te wisselen tussen activities en de simpelste om toe te passen. Voor een intent heb je niet veel code nodig. Maar we moeten wat voorbereiden voordat we deze kunnen toepassen. Hieronder is een lijstje:

- Zorg dat je in beide activities uniek eruit zien, zo kan je makkelijk onderscheiden op welke pagina je zit
- Zorg ervoor dat minimaal je main activity een button heeft en een onclicklistener, deze is nodig voor het wisselen.

Zodra de voorbereiding klaar is kan je aan de intent werken.

Binnen de onclick listener die je net hebt gemaakt schrijf je de volgende code

Intent intent = new Intent(this, TweedeActivity.class);
startActivity(intent);

Intent is de intentie code om van scherm te wisselen, dit is zowel binnen als buiten de app. Dus als je via je app de email applicatie wil opengooien met daarin het mail adres voorbereid erin dan kan dat via intent.

Met intent kan je veel meer dan alleen een nieuwe activity opzetten maar het merendeel valty buiten deze handout. Wel is het belangrijk om de context mee te geven bij de app “this” en de class van de activity die je wil starten, in dit code voorbeeld is dat TweedeActivity.

Hierna geven we aan dat deze actie uitgevoerd moet worden door startActivity te roepen met onze intentie.

Nu als je op de knop drukt zou de intent moeten afvuren en je van pagina laten wisselen.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable( $this$enableEdgeToEdge: this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v,
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.
        return insets;
    });

    Button SwitchButton = findViewById(R.id.SwitchButton);
    SwitchButton.setOnClickListener( View View->{

        Intent intent = new Intent( packageContext: this, TweedeActivity.class);
        intent.putExtra( name: "Key1",  value: "waarde");
        startActivity(intent);

    });
```

2.2.1. variabelen meegeven met intent

Als je een variabele mee moet geven met je nieuwe scherm dan kan dat door na de intent, maar voor het afvuren nog wat extra code toe te voegen.

Het gaat om deze code:

intent.putExtra("key1", "waarde");

In de extra elementen kan je waardes meegeven, dit kunnen strings, booleans, cijfers en meer zijn. Alleen objecten gaat lastiger om mee te geven op deze wijze. Het is ook inefficient om op deze wijze data over te sturen, maar kleine hoeveelheden zijn mogelijk.

Dan bij je oncreate methode van je tweede activity maak je gebruik van deze code:

Intent intent = getIntent();
String value = intent.getStringExtra("Key1");

Dit haalt de intentie op die je had gemaakt om tot die pagina te komen en laat je toe om waardes op te halen. Zorg er wel voor dat je key waarbij je hem meegeeft precies hetzelfde is, dit is hoofdletter gevoelig.

Wel belangrijk is om te weten dat welk type waarde je ophaalt, anders werkt het ophalen niet.

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable( $this$enableEdgeToEdge: this);
    setContentView(R.layout.activity_tweede);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.
        Insets systemBars = insets.getInsets(WindowInsetsComp
        v.setPadding(systemBars.left, systemBars.top, systemB
        return insets;
    });

    Intent intent = getIntent();
    String value = intent.getStringExtra( name: "Key1");
```

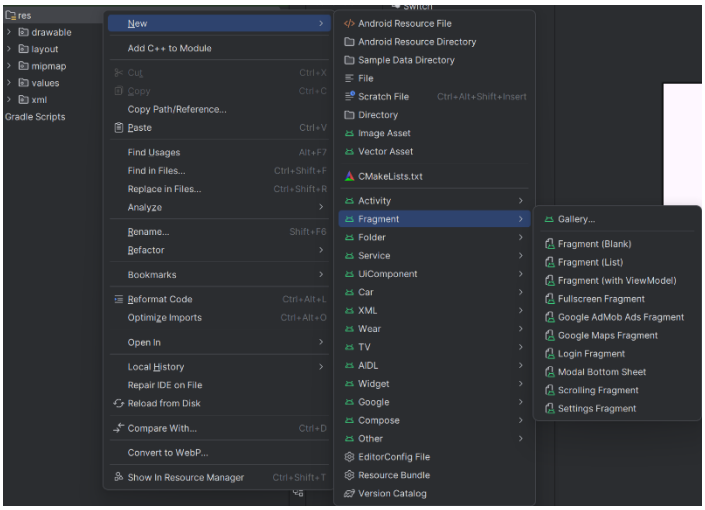
3. Fragments

Fragments worden niet standaard aangemaakt, dit is omdat sommige apps er geen noodzaak voor hebben. Veel professionele apps maken echter gebruik van fragments omdat ze vele malen handiger en sneller zijn dan activities (factor van 10X minstens).

3.1. Hoe maak je een fragment aan

Voor het maken van een fragment moet je vergelijkbare stappen maken als een activity.

- Met de rechtemuisknop klik op de res (resource) folder
- In dit menu hoever over “New”
- In dit submenu hoever over “Fragment”
- Klik op Fragment (Blank)



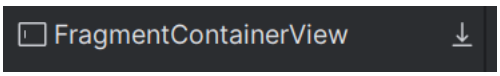
Net als activities zijn er vele templates voor verschillende fragments echter voor deze instructie is een blank fragment voldoende.

Ook hier geef je de fragment een naam die relateert aan het onderwerp en uniek is. zorg er voor dat het Java is.

3.2. Containers voor elementen

Er zijn twee versies van containers die gebruikt kan worden om fragments te bevatten, een die handgemaakt is en een die speciaal voorbereid is voor fragments. De handgemaakte container maakt gebruik van een framelayout waarin de fragments in worden opgehangen. Deze werkt heel simpel maar bevat geen quality of life functionaliteiten zoals een makkelijke te implementeren startfragment.

De tweede is een FragmentContainerView, deze moet eerst geïnstalleerd worden voor ieder project. Dit is te zien aan het download symbool naast de naam in het palette.



Na het downloaden kan deze in de activity geplaatst worden

- Voordat we de container plaatsen, zorg ervoor dat je twee button bovenin plaatst om zo makkelijker te kunnen wisselen
- Sleep de container in de activity.
- Kies je Default fragment die je als eerste visueel wil hebben.
- Zorg ervoor dat de container een breedte en lengte heeft, bij voorkeur groot genoeg om inhoud te zien.
- Zorg er ook voor dat je container een herkenbare ID heeft, deze is later nodig.

3.3. Wisselen tussen fragments

Nu dat we 1 fragment en 1 container hebben, gaan we stappen maken om te wisselen. Maak een tweede fragment aan en zorg ervoor dat er een herkenbaar verschil tussen de twee is in de xml.

Met de tweede fragment af geef beide buttons een onclick listener en een ID, een voor de eerste fragment en een voor de tweede fragment.

```
2 usages
public FragmentManager fragmentManager;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable( $this$enableEdgeToEdge: this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), ( View v,
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemB
        return insets;
    });

    Button FirstFragmentButton = findViewById(R.id.FirstFragmentButton);
    FirstFragmentButton.setOnClickListener( View View->{
        fragmentManager.beginTransaction()
                .replace(R.id.MainContainer, FirstFragment.class,  args: null)
                .addToBackStack( name: null)
                .commit();
    });

    Button SecondFragmentButton = findViewById(R.id.SecondFragmentButton);
    SecondFragmentButton.setOnClickListener( View View->{
        fragmentManager.beginTransaction()
                .replace(R.id.MainContainer, SecondFragment.class,  args: null)
                .addToBackStack( name: null)
                .commit();
    });

}
```

Binnen de oncreate van de mainactivity maken we een globale Fragment manager aan, deze gaat de jou het makkelijker te maken om te wisselen tussen de twee.

Daarna maken we de volgende code in de eerste button (Om naar fragment 1 te wisselen):

fragmentManager.beginTransaction()
    .replace(R.id.*MainContainer*, FirstFragment.class, null)
    .addToBackStack(null)
    .commit();

Dit geeft aan dat de fragment manager een nieuwe transactie moet maken (De intent van fragments).

We geven aan dat het gaat om het vervangen van een fragment met een ander binnen de MainContainer, de container die we hebben gemaakt. voor de Class van onze fragment en de null kan vervangen worden met een string als er een tag op gezet moet worden om binnen de container te zoeken naar een specifieke fragment.

.addToBackStack(null) zorgt ervoor dat je met de terug knop van android devices makkelijk terug kan klikken.

De .commit() is het in gang zetten van de transactie die je hebt gemaakt.

Voor de tweede knop moet dan uiteraard de FirstFragment.class hernoemd worden naar de SecondFragment.class

Nu kunnen we wisselen tussen de twee fragments.

Om data over te zetten kan dit op twee manieren, Android probeert het via de best practise aan te moedigen maar voor sommige apps kan de bad practise handiger zijn. Bij keuze hou je aan de best practise.

3.3.1. data overdragen – bad practise

Om data op deze wijze over te zetten moet er binnen de code van de fragment wat schoon gemaakt worden.

```java
/**
 * A simple {@link Fragment} subclass.
 * Use the {@link SecondFragment#newInstance} factory method to
 * create an instance of this fragment.
 */
public class SecondFragment extends Fragment {

    // TODO: Rename parameter arguments, choose names that match
    // the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
    2 usages
    private static final String ARG_PARAM1 = "param1";
    2 usages
    private static final String ARG_PARAM2 = "param2";

    // TODO: Rename and change types of parameters
    1 usage
    private String mParam1;
    1 usage
    private String mParam2;

    public SecondFragment() {
        // Required empty public constructor
    }

    /**
     * Use this factory method to create a new instance of
     * this fragment using the provided parameters.
     *
     * @param param1 Parameter 1.
     * @param param2 Parameter 2.
     * @return A new instance of fragment SecondFragment.
     */
    // TODO: Rename and change types and number of parameters
    1 usage
    public static SecondFragment newInstance(String param1, String param2) {
        SecondFragment fragment = new SecondFragment();
        Bundle args = new Bundle();
        args.putString(ARG_PARAM1, param1);
        args.putString(ARG_PARAM2, param2);
        fragment.setArguments(args);
        return fragment;
    }

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        if (getArguments() != null) {
            mParam1 = getArguments().getString(ARG_PARAM1);
            mParam2 = getArguments().getString(ARG_PARAM2);
        }
    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_second, container, attachToRoot: false);
    }
}
```

Naar dit ->

```java
public class SecondFragment extends Fragment {

    public SecondFragment() {
        // Required empty public constructor
    }


    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_second, container, attachToRoot: false);
    }

}
```

Daarna zetten we een extra override functie in de fragment:

```
@Override
public void onViewCreated(View view, Bundle savedInstanceState) {
   super.onViewCreated(view, savedInstanceState);
}
```

Deze zetten we ergins binnen de class. Deze onder de super.onViewCreated kunnen we code schrijven voor het initializeren van elementen binnen de fragment.

Hierna zetten we de lege constructor om naar een constructor met parameters

```
public SecondFragment(String displaytext) {
   Display = displaytext;
}
```

we maken dan een globale variabel en gaan deze benutten om informatie over te zetten.

In de oncreated zetten we de code om een textview de meegegeven tekst te laten weergeven.

```
TextView text = getView().findViewById(R.id.SecondFragmentText);
text.setText(Display);
```

Dit is heel vergelijkbaar als bij een activity echter is getView() nodig omdat we binnen een fragment werken en niet een activity.

Als laatste moeten we de transactie aanpassen, deze gaat nu ook anders.

```
SecondFragment newFragment = new SecondFragment("Hello world on the other fragment");
fragmentManager.beginTransaction()
    .replace(R.id.MainContainer, newFragment, null)
    .addToBackStack(null)
    .commit();
```

De fragment moet nu eerst aangemaakt worden en als object meegegeven worden.

Nu kan je data meegeven met de fragment bij het aanmaken. Mocht je veel data opgeslagen hebben op de main activity, dan kan je die ook meegeven aan de fragment en vanuit daar werken met public variabelen.

Belangrijke notitie:

Dit werkt NIET voor de default fragment die je bij de fragmentcontainer erin zet, deze wordt namelijk aangemaakt voordat de main activity klaar is met opzetten en dus anders crasht bij opstart.

De reden waarom dit niet best practise is, is omdat als je switcht naar een andere oriëntatie of structuur, dit als gevolgen kan hebben dat de app crasht omdat het nieuwe data niet kan inladen via de correcte routes. (Zowel activities als fragments worden met intent en transaction opnieuw ingelaten als je van oriëntatie wisselt terwijl als je via constructor wisselt deze niet meegenomen wordt vaak en dus kan leiden tot onnodige crashes.)

De reden om dit te gebruiken is voor snelle fixes en als de orientatie geen rol speelt, echter zelfs dan kunnen er enkele problemen ontstaan, hou hier rekening mee.

3.3.2. data overdragen – best practise

De juiste en correcte wijze hoef je niet slopend te werken bij de fragment inhoud, die kan blijven staan maar mag ook leeg zoals bij bad practise.

data overzetten vanaf de main activity doe je met arguments en bundles als volgt:

```
Bundle bundle = new Bundle();
bundle.putString("Key1", "Hello world from the other fragment");
SecondFragment newFragment = new SecondFragment();
newFragment.setArguments(bundle);
fragmentManager.beginTransaction()
      .replace(R.id.MainContainer, newFragment, null)
      .addToBackStack(null)
      .commit();
```

Dan ophalen doe je ook met de onViewCreated op deze wijze:

```
@Override
public void onViewCreated(View view, Bundle savedInstanceState) {
   super.onViewCreated(view, savedInstanceState);

   String display = requireArguments().getString("Key1");
   TextView text = getView().findViewById(R.id.SecondFragmentText);
   text.setText(display);
}
```

Dit zorgt voor een nette overdracht tussen de main activity en de fragment.

Dit werkt niet handig voor grote objecten die op de main activity staan, dit kan je oplossen door deze code toe te passen:

```
MainActivity activity = (MainActivity) getActivity();

String text = activity.Display;
```

Hiermee kan je data die globaal staat of functies die globaal staan gebruiken zonder de main activity te hoeven mee te geven.

3.4. Gevorderd – Layeren

Layeren is iets wat je kan doen bij fragments maar niet bij activities. Dit laat je toe om met doorzichtige achtergronden twee lagen over elkaar te zetten. Zorg er wel voor dat de achtergrond van de fragment daadwerkelijk doorzichtig is.

Om dit te bereiken plaatsen we bij de transactie inplaats van .replace een .add functie:

```
fragmentManager.beginTransaction()
        .add(R.id.MainContainer, LayerFragment.class, null)
        .commit();
```

Meer is niet nodig om de laag toe te voegen

Om deze layering te verwijderen gebruiken we .remove. .replace kan ook maar dan worden beide fragments vervangen en als maar 1 verwijderd hoeft te worden is dit mogelijk overdreven.

Voor de .remove moeten we echter wel de specifieke fragment als object hebben om te verwijderen. Om deze fragment te vinden geven we de fragment bij het aanmaken een tag/naam. Dan wordt de .add zo:

```
.add(R.id.MainContainer, LayerFragment.class, "laagfragment")
```

Nu kunnen we zoeken naar de laag en het verwijderen als volgt:

```
Fragment existingLayer = fragmentManager.findFragmentByTag("laagfragment");
  if(existingLayer != null){
     fragmentManager.beginTransaction().remove(existingLayer).commit();
  }
```

Nu hebben we de laag verwijdert.

4. Hand-out afsluiting

Nu dat je de hand-out doorlopen hebt, probeer de oefenopdrachten te maken met wat, deze voorbeeld projecten staan klaar op de GitHub. Kom je er niet uit, geen probleem de uitwerkingen staan ook op de GitHub.

Nadat die zijn gelukt ben je klaar om nieuwe pagina's te maken voor je eigen applicatie!